

WEEK-10

TASK-1: Investigate inode structures using commands such as ls -li, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

```
root@LAPTOP-RJFDBEBT:~# mkdir inode_test
cd inode_test
root@LAPTOP-RJFDBEBT:~/inode_test# nano original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# ls -li
1932 original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# ls -li
total 4
1932 -rw-r--r-- 1 root root 61 Sep 27 14:04 original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# stat original.txt
  File: original.txt
  Size: 61          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 1932         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 0/   root)   Gid: ( 0/   root)
Access: 2026-09-27 14:04:48.112031197 +0000
Modify: 2026-09-27 14:04:48.114622243 +0000
Change: 2026-09-27 14:04:48.114622243 +0000
 Birth: 2026-09-27 14:04:48.112031197 +0000
root@LAPTOP-RJFDBEBT:~/inode_test# ln original.txt hardlink.txt
root@LAPTOP-RJFDBEBT:~/inode_test# ls -li
total 8
1932 -rw-r--r-- 2 root root 61 Sep 27 14:04 hardlink.txt
1932 -rw-r--r-- 2 root root 61 Sep 27 14:04 original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# stat original.txt
  File: original.txt
  Size: 61          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 1932         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 0/   root)   Gid: ( 0/   root)
Access: 2026-09-27 14:04:48.112031197 +0000
Modify: 2026-09-27 14:04:48.114622243 +0000
Change: 2026-09-27 14:04:48.114622243 +0000
 Birth: 2026-09-27 14:04:48.112031197 +0000
root@LAPTOP-RJFDBEBT:~/inode_test# stat hardlink.txt
  File: hardlink.txt
  Size: 61          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 1932         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 0/   root)   Gid: ( 0/   root)
Access: 2026-09-27 14:04:48.112031197 +0000
Modify: 2026-09-27 14:04:48.114622243 +0000
Change: 2026-09-27 14:04:48.114622243 +0000
 Birth: 2026-09-27 14:04:48.112031197 +0000
root@LAPTOP-RJFDBEBT:~/inode_test# ln -s original.txt symlink.txt
root@LAPTOP-RJFDBEBT:~/inode_test# ls -li
total 8
1932 -rw-r--r-- 2 root root 61 Sep 27 14:04 hardlink.txt
1938 lrwxrwxrwx 1 root root 12 Sep 27 14:10 symlink.txt -> original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# ls -li
lrwxrwxrwx 1 root root 12 Sep 27 14:10 symlink.txt -> original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 1938         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 0/   root)   Gid: ( 0/   root)
Access: 2026-09-27 14:11:08.178680181 +0000
Modify: 2026-09-27 14:10:53.979686850 +0000
Change: 2026-09-27 14:10:53.979686850 +0000
 Birth: 2026-09-27 14:10:53.979686850 +0000
root@LAPTOP-RJFDBEBT:~/inode_test# find . -inum 1932
./original.txt
./hardlink.txt
root@LAPTOP-RJFDBEBT:~/inode_test# find . -ls
 1914    4 drwxr-xr-x  2 root    root      4096 Sep 27 14:18 .
 1930    0 lrwxrwxrwx  1 root    root      12 Sep 27 14:10 ./symlink.txt -> original.txt
 1932    4 -rw-r--r--  2 root    root      61 Sep 27 14:04 ./hardlink.txt
 1932    4 -rw-r--r--  2 root    root      61 Sep 27 14:04 ./original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# cat original.txt
This is the original file.
root@LAPTOP-RJFDBEBT:~/inode_test# cat hardlink.txt
This is the original file.
root@LAPTOP-RJFDBEBT:~/inode_test# cat symlink.txt
This is the original file.
We are studying inode allocation.
root@LAPTOP-RJFDBEBT:~/inode_test# rm original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# ls -li
total 4
1932 -rw-r--r-- 1 root root 61 Sep 27 14:04 hardlink.txt
1938 lrwxrwxrwx 1 root root 12 Sep 27 14:10 symlink.txt -> original.txt
root@LAPTOP-RJFDBEBT:~/inode_test# cat hardlink.txt
This is the original file.
We are studying inode allocation.
root@LAPTOP-RJFDBEBT:~/inode_test# cat symlink.txt
cat: symlink.txt: No such file or directory
root@LAPTOP-RJFDBEBT:~/inode_test# stat original.txt
stat: cannot statx 'original.txt': No such file or directory
root@LAPTOP-RJFDBEBT:~/inode_test#
```

TASK-2: Design a program using mmap() to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and Implementation complexity.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <sys/mman.h>
```

```
#include <sys/stat.h>
```

```
#include <time.h>
```

```
#include <string.h>
```

```
void traditional_copy(const char *source, const char *destination)
```

```
{
```

```
    int src, dest;
```

```
    char buffer[BUFFER_SIZE];
```

```
    ssize_t bytes_read, bytes_written;
```

```

src = open(source, O_RDONLY);
if (src == -1)
{
    perror("open source");
    exit(1);
}
dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (dest == -1)
{
    perror("open destination");
    close(src);
    exit(1);
}
while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
{
    bytes_written = write(dest, buffer, bytes_read);
    if (bytes_written != bytes_read)
    {
        perror("write");
        close(src);
        close(dest);
        exit(1);
    }
}

close(src);
close(dest);
}

void mmap_copy(const char *source, const char *destination)
{
    int src, dest;
    struct stat file_info;
    char *src_map, *dest_map;

```

```
size_t file_size;

src = open(source, O_RDONLY);

if (src == -1)
{
    perror("open source");
    exit(1);
}

if (fstat(src, &file_info) == -1)
{
    perror("fstat");
    close(src);
    exit(1);
}

file_size = file_info.st_size;

if (file_size == 0)
{
    close(src);

    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    close(dest);
    return;
}

src_map = mmap(NULL, file_size, PROT_READ, MAP_PRIVATE, src, 0);

if (src_map == MAP_FAILED)
{
    perror("mmap source");
    close(src);
    exit(1);
}

dest = open(destination, O_RDWR | O_CREAT | O_TRUNC, 0644);

if (dest == -1)
{
    perror("open destination");
    munmap(src_map, file_size);
```

```

    close(src);
    exit(1);
}
if (ftruncate(dest, file_size) == -1)
{
    perror("ftruncate");
    munmap(src_map, file_size);
    close(src);
    close(dest);
    exit(1);
}
dest_map = mmap(NULL, file_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED, dest, 0);

if (dest_map == MAP_FAILED)
{
    perror("mmap destination");
    munmap(src_map, file_size);
    close(src);
    close(dest);
    exit(1);
}
memcpy(dest_map, src_map, file_size);
if (msync(dest_map, file_size, MS_SYNC) == -1)
{
    perror("msync");
}
munmap(src_map, file_size);
munmap(dest_map, file_size);
close(src);
close(dest);
}

```

```

int main()
{
    const char *source = "source.txt";
    const char *traditional_file = "traditional.txt";
    const char *mmap_file = "mmap.txt";

    clock_t start, end;

    double traditional_time;

    double mmap_time;

    printf("File I/O Performance Comparison\n");
    printf("-----\n");

    start = clock();

    traditional_copy(source, traditional_file);

    end = clock();

    traditional_time =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Traditional read/write copy completed.\n");
    printf("Time: %f seconds\n", traditional_time);

    start = clock();

    mmap_copy(source, mmap_file);

    end = clock();

    mmap_time =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Memory-mapped copy completed.\n");
    printf("Time: %f seconds\n", mmap_time);

    printf("\nPerformance Comparison\n");
    printf("-----\n");

    if (traditional_time < mmap_time)
        printf("Traditional read/write was faster in this run.\n");
    else if (mmap_time < traditional_time)
        printf("Memory-mapped I/O was faster in this run.\n");
    else
        printf("Both methods took approximately the same time.\n");

    return 0;
}

```

}

```
root@LAPTOP-RJFD8EBT:~/inode_test# nano mmap_compare.c
root@LAPTOP-RJFD8EBT:~/inode_test# nano source.txt
root@LAPTOP-RJFD8EBT:~/inode_test# gcc mmap_compare.c -o mmap_compare
root@LAPTOP-RJFD8EBT:~/inode_test# ./mmap_compare
File I/O Performance Comparison
-----
Traditional read/write copy completed.
Time: 0.000149 seconds
Memory-mapped copy completed.
Time: 0.000566 seconds

Performance Comparison
-----
Traditional read/write was faster in this run.
root@LAPTOP-RJFD8EBT:~/inode_test# ls -l
total 44
-rw-r--r-- 1 root root    61 Sep 27 14:04 hardlink.txt
-rw-r--r-- 1 root root   109 Sep 27 14:29 mmap.txt
-rwxr-xr-x 1 root root 16680 Sep 27 14:28 mmap_compare
-rw-r--r-- 1 root root   4215 Sep 27 14:21 mmap_compare.c
-rw-r--r-- 1 root root    109 Sep 27 14:27 source.txt
lrwxrwxrwx 1 root root    12 Sep 27 14:10 symlink.txt -> original.txt
-rw-r--r-- 1 root root    109 Sep 27 14:29 traditional.txt
root@LAPTOP-RJFD8EBT:~/inode_test# cat traditional.txt
This is a test file for memory mapped I/O.
We are comparing mmap with traditional read and write operations.
root@LAPTOP-RJFD8EBT:~/inode_test# cat mmap.txt
This is a test file for memory mapped I/O.
We are comparing mmap with traditional read and write operations.
root@LAPTOP-RJFD8EBT:~/inode_test# cmp source.txt traditional.txt
root@LAPTOP-RJFD8EBT:~/inode_test# cmp source.txt mmap.txt
root@LAPTOP-RJFD8EBT:~/inode_test#
```